

Classification, Arbres

& Régression Polynomiale

Du modèle linéaire aux modèles flexibles — quand changer d'outil

#	Objectif	Compétence
1	Différencier régression et classification selon la nature de la cible	C3.3
2	Formuler un problème de classification binaire avec la régression logistique	C3.3
3	Comprendre la fonction sigmoïde et la fonction coût entropie croisée	C3.3
4	Évaluer un modèle de classification : matrice de confusion, métriques, ROC/AUC	C3.4
5	Implémenter une régression logistique avec scikit-learn	C3.3
6	Utiliser les arbres de décision pour la régression	C3.3
7	Réaliser une régression polynomiale et gérer le surapprentissage	C3.3

1

Régression vs Classification

Choisir le bon paradigme selon la nature de la variable cible

1.1 Deux paradigmes, un seul principe

En ML supervisé, tout commence par la même question : quelle est la nature de la variable cible y ? La réponse conditionne le choix de l'algorithme, de la fonction coût et des métriques d'évaluation.



Régression

- y est une valeur continue : prix, température, salaire, score
- Le modèle prédit un nombre réel : $\hat{y} \in \mathbb{R}$
- Exemples : prix d'une maison (250 000 €), consommation électrique (3.4 kWh)
- Métriques : RMSE, MAE, R^2
- Algorithmes : régression linéaire, Ridge, Lasso, arbres de régression



Classification

- y est une classe discrète : spam/non-spam, malade/sain, A/B/C
- Le modèle prédit une classe (et souvent une probabilité d'appartenance)
- Exemples : email est spam (oui/non), tumeur est maligne (oui/non)
- Métriques : accuracy, précision, rappel, F1, AUC-ROC
- Algorithmes : régression logistique, arbres de décision, SVM, KNN

1.2 Pourquoi on ne peut pas utiliser la régression linéaire pour classifier

Question naturelle : puisqu'on peut encoder les classes (0 = non-spam, 1 = spam), pourquoi ne pas appliquer directement une régression linéaire ?

✗ Trois problèmes fondamentaux de la régression linéaire pour la classification

Problème 1 — Prédiction hors $[0,1]$: la régression linéaire peut prédire $\hat{y} = 1.8$ ou $\hat{y} = -0.3$, ce qui n'a aucun sens comme probabilité.

Problème 2 — Sensibilité aux outliers : un seul point très éloigné peut faire basculer la frontière de décision de façon catastrophique.

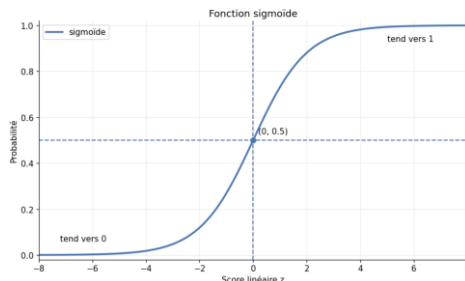
Problème 3 — Hypothèses violées : les résidus d'une régression linéaire sur une cible binaire ne sont jamais normaux ni homoscedastiques.

La solution est d'utiliser une fonction qui mappe les sorties d'une combinaison linéaire vers l'intervalle $[0,1]$. C'est précisément ce que fait la **régression logistique** via la fonction sigmoïde.

Régression logistique

De la combinaison linéaire à la probabilité de classe

2.1 La fonction sigmoïde — transformer $\mathbb{R}$ en probabilité



La régression logistique ne prédit pas directement $y \in \{0,1\}$. Elle prédit la probabilité que $y = 1$, c'est-à-dire $P(y = 1 | x)$. Pour cela, elle passe la combinaison linéaire $z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$ à travers la fonction sigmoïde.

Fonction sigmoïde $\sigma(z)$

$$P(y=1 | x) = \sigma(z) = 1 / (1 + e^{-z}) \quad \text{où } z = \beta_0 + \sum \beta_j x_j$$

Propriétés : $\sigma(z) \in (0, 1)$ pour tout z réel · $\sigma(0) = 0.5$ · $\sigma(+\infty) = 1$ · $\sigma(-\infty) = 0$

La sigmoïde ne vient pas d'un choix "au hasard" ni seulement d'un choix graphique. Elle est utilisée en **régression logistique** parce qu'elle remplit plusieurs propriétés très utiles **à la fois**.

1) Il faut transformer une sortie linéaire en probabilité

Un modèle linéaire calcule d'abord :

$$z = \beta_0 + \sum \beta_j x_j$$

Mais (z) peut valoir :

- (-5)
- (0.7)
- (12)

Or une probabilité doit être entre **0 et 1**.

La sigmoïde fait exactement cela :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Elle prend n'importe quel réel et le transforme en une valeur dans **]0,1[**.

2) Elle est monotone

Quand (z) augmente, la probabilité augmente aussi.

Donc :

- score linéaire plus grand $\rightarrow$ probabilité plus grande de classe 1
- score plus petit $\rightarrow$ probabilité plus faible

C'est cohérent pour un classifieur.

3) Elle garde une frontière de décision linéaire

Avec la sigmoïde :

$$P(Y = 1 | X) = \sigma(z)$$

et décider "classe 1 si probabilité > 0.5 " revient à décider :

$$z > 0$$

Donc la frontière de décision reste :

$$\beta_0 + \sum \beta_j x_j = 0$$

C'est très pratique :

- on garde la simplicité d'un modèle linéaire
- mais on obtient une sortie probabiliste valide

4) Elle a une très belle interprétation statistique

La vraie force de la régression logistique, ce n'est pas juste la forme en S.

Avec la sigmoïde, on a :

$$\log \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Autrement dit, **le log-odds** est linéaire.

C'est très utile car :

- les coefficients ont une interprétation claire
- une augmentation d'une variable change les **odds** de façon multiplicative

5) Elle est cohérente avec le modèle Bernoulli

En classification binaire, la cible suit naturellement une loi **Bernoulli**.

La sigmoïde apparaît naturellement quand on modélise une variable binaire avec :

- une vraisemblance Bernoulli
- un lien logit

Donc la régression logistique n'est pas juste :

“on prend une courbe en S parce qu'elle a l'air bien”

C'est un modèle statistique cohérent.

6) L'optimisation est pratique

Avec la sigmoïde + la log-loss, on obtient un problème d'optimisation bien maîtrisé, souvent convexe pour la régression logistique classique.

Cela veut dire :

- apprentissage stable
- calcul efficace
- solution bien définie

Pourquoi pas une autre fonction ?

En réalité, **on pourrait** utiliser une autre fonction, à condition qu'elle :

- transforme (R vers $([0,1])$)
- soit monotone
- soit exploitable mathématiquement

Donc la vraie réponse est :

ce n'est pas que la sigmoïde est la seule possible ; c'est qu'elle est l'une des plus naturelles, interprétables et pratiques.

Comment lire la sigmoïde

Valeur de z	$\sigma(z)$	Interprétation
---------------	-------------	----------------

$z = 0$	0.500	Incertitude maximale — le modèle hésite entre les deux classes
$z = 2$	0.880	88% de probabilité que $y = 1$ — classifié positif avec confiance
$z = -2$	0.119	12% de probabilité que $y = 1$ — classifié négatif
$z = 4$	0.982	98% — grande confiance dans la classe positive
$z = -4$	0.018	2% — grande confiance dans la classe négative

De la probabilité à la décision : le seuil de classification

La probabilité seule ne suffit pas pour décider d'une classe. On fixe un seuil θ (threshold) :

Règle de décision

**Si $P(y=1|x) \geq \theta \rightarrow$ classe prédite = 1 Sinon $\rightarrow$
classe prédite = 0**

Par défaut $\theta = 0.5$. Mais selon le contexte, on peut le modifier (voir Partie 3 : compromis précision/rappel).

💡 Analogie médicale : test de dépistage

Un médecin mesure z = combinaison de marqueurs biologiques d'un patient.

$\sigma(z) = 0.73 \rightarrow$ probabilité de 73% que le patient soit malade.

Avec $\theta = 0.5$: le patient est classifié comme malade (positif).

Si le médecin abaisse θ à 0.3 : il classe davantage de patients comme malades $\rightarrow$ moins de cas manqués, mais plus de fausses alarmes.

2.2 La frontière de décision

La régression logistique trouve une frontière de décision linéaire dans l'espace des features. En 2D, c'est une droite. En 3D, un plan. En général, un hyperplan.

Equation de la frontière : $\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0 \rightarrow$ c'est l'ensemble des points où $\sigma(z) = 0.5$

D'un côté : $z > 0 \rightarrow \sigma(z) > 0.5 \rightarrow$ classe prédite = 1

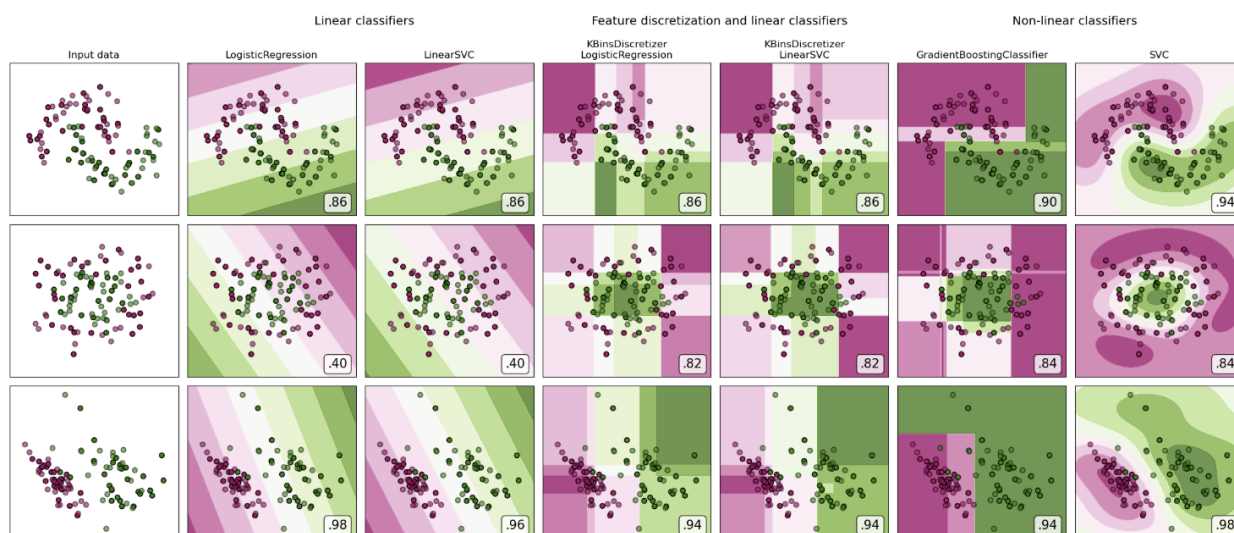
De l'autre : $z < 0 \rightarrow \sigma(z) < 0.5 \rightarrow$ classe prédite = 0

⚠ Limitation fondamentale de la régression logistique

La frontière de décision est toujours LINÉAIRE. Si les classes ne sont pas séparables par un hyperplan, la régression logistique ne peut pas les séparer.

Exemple : données en cercles concentriques (classe 0 au centre, classe 1 autour) $\rightarrow$ impossible pour la régression logistique.

Solutions : features polynomiales (transformer x_1, x_2 en $x_1^2, x_2^2, x_1 x_2 \dots$) ou utiliser un modèle non-linéaire (arbres, SVM, réseaux de neurones).



2.3 La fonction coût — entropie croisée (Log Loss)

On ne peut pas utiliser la MSE avec une cible binaire — la fonction résultante n'est pas convexe et la descente de gradient peut se bloquer dans des minima locaux.

La fonction coût adaptée à la classification binaire est l'entropie croisée (aussi appelée Log Loss) :

Entropie croisée binaire

$$J(\beta) = -(1/n) \cdot \sum [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$$

où $\hat{y}_i = \sigma(\beta_0 + \sum \beta_j x_{ij})$ est la probabilité prédite pour l'observation i .

Pourquoi cette formule ? Décryptage

Cas	Formule active	Comportement
$y_i = 1$ (vrai positif) et $\hat{y}_i \approx 1$	$-\log(\hat{y}_i) \approx -\log(1) = 0$	Coût quasi nul — bonne prédiction ✓
$y_i = 1$ (vrai positif) et $\hat{y}_i \approx 0$	$-\log(\hat{y}_i) \rightarrow +\infty$	Coût énorme — grave erreur ✗
$y_i = 0$ (vrai négatif) et $\hat{y}_i \approx 0$	$-\log(1-\hat{y}_i) \approx -\log(1) = 0$	Coût quasi nul — bonne prédiction ✓
$y_i = 0$ (vrai négatif) et $\hat{y}_i \approx 1$	$-\log(1-\hat{y}_i) \rightarrow +\infty$	Coût énorme — grave erreur ✗

Propriété clé : l'entropie croisée est convexe en β — la descente de gradient converge toujours vers l'optimum global (pas de minimum local).

🔗 Comparaison avec la MSE

MSE = moyenne des carrés des erreurs. Convexe pour la régression linéaire mais pas pour la logistique.

Entropie croisée = mesure de la « surprise » liée à une mauvaise prédiction probabiliste. Très grande quand on est très sûr et très faux.

Règle : pour une cible continue → MSE/RMSE. Pour une cible binaire ou multi-classe → entropie croisée.

2.4 Régularisation de la régression logistique

Comme la régression linéaire, la régression logistique peut overfitter si elle a beaucoup de features ou si les classes sont presque séparables linéairement (les coefficients tendent vers l'infini).

Paramètre C dans sklearn : $C = 1/\lambda$ est l'inverse du paramètre de régularisation. Un **C petit** = forte régularisation. Un **C grand** = faible régularisation (proche de OLS).

penalty='l2' (Ridge logistique) — défaut

- Réduit tous les coefficients vers 0 sans les annuler
- Stable face à la multicolinéarité
- Recommandé par défaut
- C optimal : à trouver par cross-validation

penalty='l1' (Lasso logistique)

- Annule exactement certains coefficients
- Sélection automatique de features
- Utile si on suspecte beaucoup de features irrelevantes
- Moins stable si features très corrélées

2.5 Implémentation avec scikit-learn

Voici le workflow complet d'une régression logistique binaire, de la préparation des données à l'interprétation.

```
# — Étape 1 : Importer et préparer —————
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, roc_auc_score

# — Étape 2 : Split + Pipeline —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y) # stratify !

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('lr', LogisticRegression(C=1.0, penalty='l2', max_iter=1000))
])

# — Étape 3 : Entraîner —————
pipe.fit(X_train, y_train)

# — Étape 4 : Prédire —————
y_pred = pipe.predict(X_test) # classes prédites
y_pred_proba = pipe.predict_proba(X_test)[:,1] # probabilités P(y=1)

# — Étape 5 : Évaluer —————
print(classification_report(y_test, y_pred))
print('AUC-ROC :', roc_auc_score(y_test, y_pred_proba))
```

⚠ Toujours utiliser stratify=y dans train_test_split pour la classification

Sans stratify, si votre dataset est déséquilibré (ex : 5% de positifs), le split aléatoire peut mettre tous les positifs dans le train ou le test.

stratify=y garantit que la proportion de chaque classe est préservée dans le train ET dans le test.

Exemple : 100 positifs / 900 négatifs → train : 80 positifs / 720 négatifs · test : 20 positifs / 180 négatifs.

Lire et interpréter les coefficients

Les coefficients d'une régression logistique s'interprètent via les log-odds (rapport de cotes) :

Interprétation des coefficients

$$\exp(\beta_j) = \text{odds ratio de } x_j$$

Si $\beta_j = 0.5 \rightarrow \exp(0.5) \approx 1.65 \rightarrow +1$ unité de x_j multiplie les chances d'être en classe 1 par 1.65 (augmentation de 65%), toutes autres features constantes.

3

Évaluer un classifieur

Matrice de confusion, métriques spécialisées, courbe ROC et AUC

3.1 La matrice de confusion — la base de tout

L'accuracy (taux de bonnes prédictions) est la métrique la plus simple mais aussi la plus trompeuse. Sur un dataset déséquilibré (99% de négatifs), un modèle qui prédit toujours « négatif » a une accuracy de 99% — et est parfaitement inutile.

La matrice de confusion donne une image complète des erreurs commises par le classifieur.

	Prédit : POSITIF (1)	Prédit : NÉGATIF (0)	Total réel
Réel : POSITIF (1)	TP Vrai Positif Malade, prédit malade	FN Faux Négatif Malade, prédit sain	TP + FN
Réel : NÉGATIF (0)	FP Faux Positif Sain, prédit malade	TN Vrai Négatif Sain, prédit sain	FP + TN

3.2 Les quatre métriques essentielles

Métrique	Formule	Interprétation	Utiliser quand...
Accuracy	$(TP+TN) / \text{Total}$	Taux global de bonnes prédictions	Dataset équilibré (50/50)
Précision	$TP / (TP+FP)$	Parmi les prédits positifs, combien le sont vraiment ?	Fausse alarmes coûteuses (spam : mieux rater un spam que supprimer un vrai email)
Rappel	$TP / (TP+FN)$	Parmi les vrais positifs, combien détecte-t-on ?	Manquer un positif coûteux (cancer : mieux sur-diagnostiquer)
F1-Score	$2 \cdot \text{Précision} \cdot \text{Rappel} / (P+R)$	Moyenne harmonique entre précision et rappel	Dataset déséquilibré, compromis précision/rappel

Le compromis Précision / Rappel

Précision et rappel sont en tension permanente. Augmenter l'un revient souvent à diminuer l'autre. Le seuil θ contrôle ce compromis :

Abaissier θ (ex : 0.3) - Plus de cas classifiés positifs - Rappel augmente (on manque moins de vrais positifs) - Précision diminue (plus de faux positifs) - Utile : détection du cancer, fraude critique	Augmenter θ (ex : 0.7) - Moins de cas classifiés positifs - Précision augmente (moins de faux positifs) - Rappel diminue (on rate plus de vrais positifs) - Utile : filtre anti-spam, recommandation commerciale
---	--

3.3 La courbe ROC et l'AUC

La courbe ROC (Receiver Operating Characteristic) visualise la performance d'un classifieur pour TOUS les seuils θ possibles, sans en choisir un a priori.

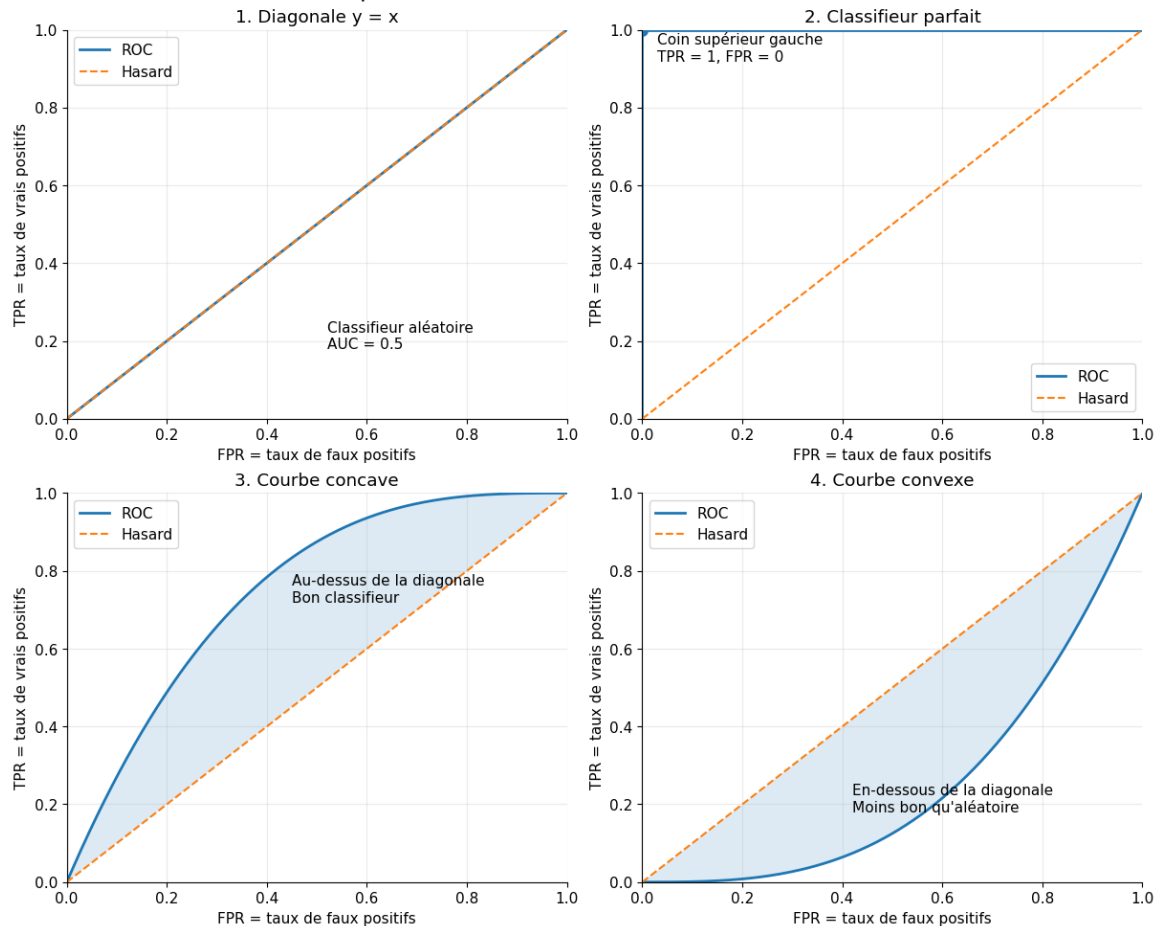
Construction de la courbe ROC

Axe X : Taux de Faux Positifs (FPR) = $FP / (FP + TN) = 1 - \text{Spécificité}$

Axe Y : Taux de Vrais Positifs (TVP) = $TP / (TP + FN) = \text{Rappel} = \text{Sensibilité}$

On trace : pour chaque valeur de θ de 0 à 1, on calcule (FPR, TVP) et on le place sur le graphique.

Interprétation de différents cas de courbe ROC



Courbe ROC	Description	Interprétation
Diagonale y=x	FPR = TVP pour tout θ	Classifieur aléatoire — équivalent à un tirage au sort. AUC = 0.5
Coin supérieur gauche	TVP = 1, FPR = 0 pour un θ	Classifieur parfait — détecte tous les positifs sans faux positif. AUC = 1.0
Courbe concave	Au-dessus de la diagonale	Bon classifieur — mieux qu'aléatoire. Plus la courbe est haute, mieux c'est
Courbe convexe	En-dessous de la diagonale	Mauvais classifieur — moins bon qu'aléatoire (inverser les prédictions !)

L'AUC — Area Under the Curve

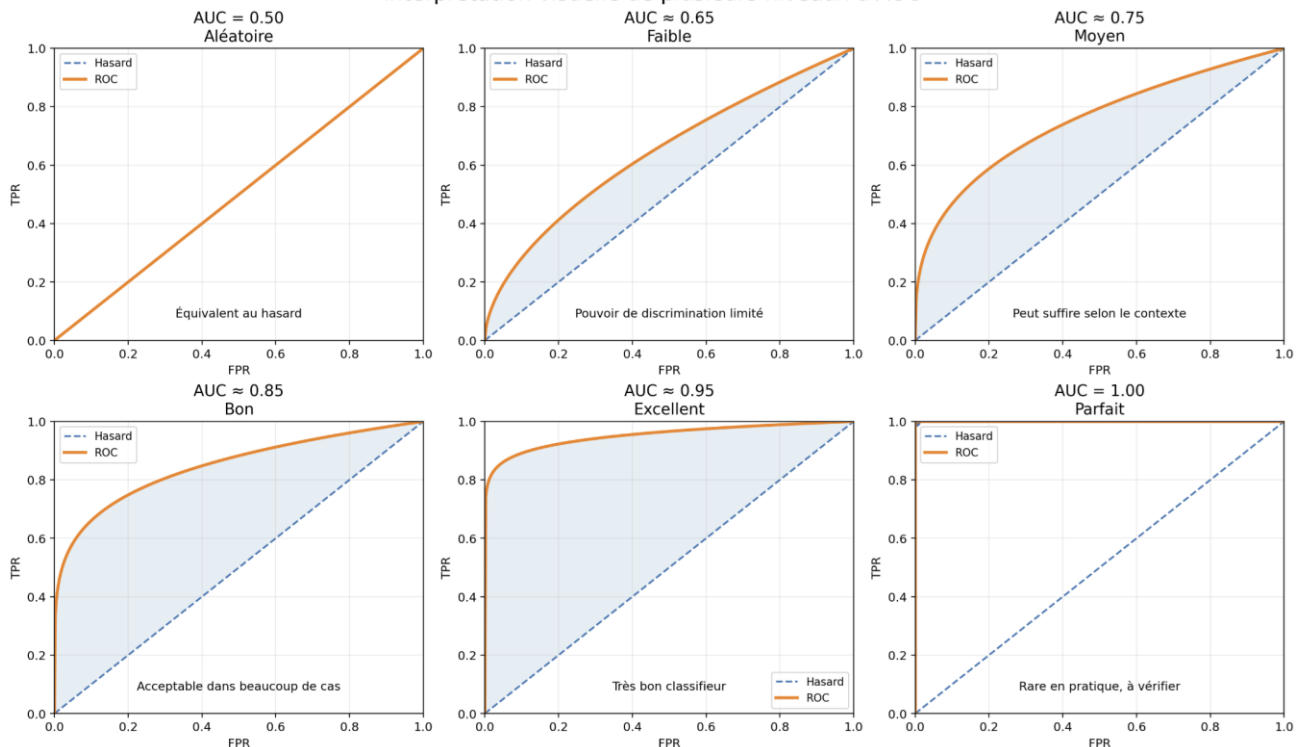
L'AUC est l'aire sous la courbe ROC. C'est la métrique de choix pour évaluer un classifieur indépendamment du seuil.

Interprétation probabiliste de l'AUC

$$\text{AUC} = P(\text{score}(\text{positif}) > \text{score}(\text{négatif}))$$

L'AUC est la probabilité que le modèle attribue un score plus élevé à un vrai positif qu'à un vrai négatif, pris au hasard. AUC = 0.85 → 85% du temps, le modèle distingue bien.

Interprétation visuelle de plusieurs niveaux d'AUC



AUC	Interprétation	Commentaire
AUC = 1.0	Classifieur parfait	Rare — suspect si atteint, penser au data leakage
AUC > 0.9	Excellent	Très bon classifieur
0.8 < AUC ≤ 0.9	Bon	Acceptable pour la plupart des cas métier
0.7 < AUC ≤ 0.8	Moyen	Peut suffire selon le contexte
AUC ≤ 0.7	Faible	À améliorer avant tout déploiement
AUC = 0.5	Aléatoire	Équivalent à un tirage au sort — modèle inutile

📊 Quand utiliser ROC/AUC vs Précision/Rappel ?

ROC/AUC : quand les deux classes sont à peu près équilibrées. Donne une vision globale indépendante du seuil.

Courbe Précision/Rappel : quand le dataset est fortement déséquilibré (1% de positifs). La ROC peut être trompeuse sur ces cas.

Exemple : détection de fraude (0.1% de fraude). Un modèle peut avoir AUC = 0.90 mais Précision = 0.05 (19 fausses alarmes pour 1 vraie fraude détectée).

Arbres de décision pour la régression

Un modèle non-linéaire basé sur des règles

4.1 Principe : diviser pour mieux prédire

Un arbre de décision découpe l'espace des features en régions rectangulaires et prédit la moyenne de y dans chaque région. C'est le premier modèle non-linéaire du cours — il n'y a pas de $\beta_0 + \beta_1 x$.

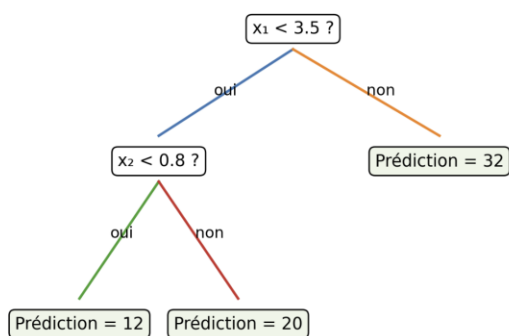
💡 Comment un arbre de régression prédit-il ?

Étape 1 — Construction : l'arbre divise récursivement les données. À chaque nœud, il choisit la feature et le seuil qui minimisent la MSE dans les deux sous-groupes.

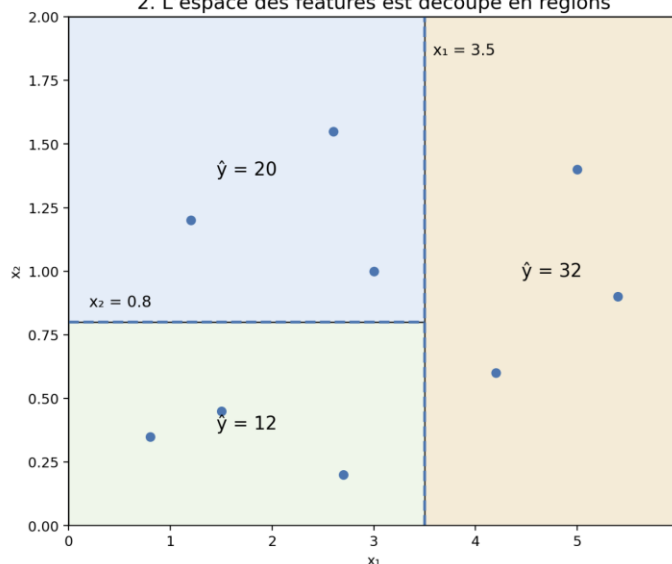
Étape 2 — Prédiction : pour un nouveau point x , on descend l'arbre en répondant aux questions ($x_1 > 3.5$? $x_2 < 0.8$?). On arrive à une feuille.

Étape 3 — Valeur prédite : la prédiction est la MOYENNE des y d'entraînement tombés dans cette feuille.

1. L'arbre pose des questions successives



2. L'espace des features est découpé en régions



4.2 Paramètres clés : contrôler la complexité

Paramètre (sklearn)	Effet	Valeur par défaut	Recommandation
max_depth	Profondeur maximale de l'arbre	None (illimité)	Commencer à 3–5. Plus profond = plus d'overfitting
min_samples_split	Nb min d'observations pour diviser un nœud	2	Augmenter sur petits datasets (5–10)
min_samples_leaf	Nb min d'observations dans une feuille	1	Très utile : 5–20 selon la taille du dataset
max_features	Nb de features considérées à chaque split	None (toutes)	Réduire pour randomiser (utile dans les forêts)

4.3 Overfitting des arbres — le problème principal

Un arbre sans contrainte va créer autant de feuilles que d'observations d'entraînement. Chaque feuille contient un seul point → MSE train = 0 → modèle mémorisé → MSE test catastrophique.

Arbre non contrôlé (max_depth=None)

- MSE train ≈ 0 (chaque feuille = 1 observation)
- MSE test très élevé — overfitting maximal
- Frontières de décision extrêmement découpées
- Ne généralise pas sur de nouvelles données

Arbre avec max_depth=3

- MSE train plus élevé (sous-optimal sur le train)
- MSE test nettement amélioré — meilleure généralisation
- Frontières lisibles et interprétables
- Modèle facilement explicable à un expert métier

```
# Arbre de décision pour la régression
from sklearn.tree import DecisionTreeRegressor, export_text

# Modèle contrôlé
arbre = DecisionTreeRegressor(max_depth=3, min_samples_leaf=5, random_state=42)
arbre.fit(X_train, y_train)

# Évaluation
rmse_train = np.sqrt(mean_squared_error(y_train, arbre.predict(X_train)))
rmse_test = np.sqrt(mean_squared_error(y_test, arbre.predict(X_test)))

# Visualiser les règles de l'arbre (très utile pour l'interprétation)
print(export_text(arbre, feature_names=list(X.columns)))

# Importance des features
importances = pd.Series(arbre.feature_importances_, index=X.columns)
importances.sort_values().plot(kind='barh')
```

Avantages et limites des arbres de décision**☑ Avantages**

- Interprétable : on peut lire les règles de décision
- Robuste aux outliers et aux features non standardisées
- Capture des relations non-linéaires et des interactions
- Fonctionne avec des features catégorielles
- Pas de normalisation nécessaire

⚠ Limites

- Très sensible à l'overfitting sans contraintes
- Instable : un seul nouveau point peut changer l'arbre entier
- Frontières de décision rectangulaires — pas optimal pour des relations diagonales
- Performance souvent inférieure à Ridge/Lasso sur données linéaires

Régression polynomiale

Capturer les non-linéarités sans changer d'algorithme

5.1 L'idée : enrichir les features, pas le modèle

La régression polynomiale n'est pas un nouvel algorithme. C'est une technique de feature engineering qui transforme les features originales en ajoutant des puissances et des interactions, puis applique une régression linéaire sur ces nouvelles features.

Régression polynomiale de degré d sur une feature x

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_d x^d$$

Le modèle reste linéaire en β — on peut toujours utiliser OLS, Ridge, Lasso. Seules les features ont changé.

💡 Analogie : changer de référentiel

La relation $y = x^2$ semble non-linéaire. Mais si on pose $z = x^2$, alors $y = z$ est parfaitement linéaire en z .

La régression polynomiale fait exactement ça : elle crée $z_1 = x$, $z_2 = x^2$, $z_3 = x^3$... et fait une régression linéaire dans ce nouvel espace.

C'est valide pour n'importe quelle transformation : $\log(x)$, $\sqrt{x}$, $x_1 \cdot x_2$ (interaction), etc.

5.2 En pratique : PolynomialFeatures dans sklearn

La classe PolynomialFeatures génère automatiquement toutes les puissances et interactions jusqu'au degré demandé.

Degré	Features générées à partir de (x_1, x_2)	Nb total de features
1 (linéaire)	x_1, x_2	2
2	$1, x_1, x_2, x_1^2, x_1 x_2, x_2^2$	6
3	$1, x_1, x_2, x_1^2, x_1 x_2, x_2^2, x_1^3, x_1^2 x_2, x_1 x_2^2, x_2^3$	10
d (général)	—	$C(p+d, d) = (p+d)! / (p! d!)$

```
# Régression polynomiale avec Pipeline
from sklearn.preprocessing import PolynomialFeatures

degree = 3 # À choisir par cross-validation
pipe_poly = Pipeline([
    ('poly', PolynomialFeatures(degree=degree, include_bias=False)),
    ('scaler', StandardScaler()), # Toujours standardiser APRÈS poly
    ('lr', Ridge(alpha=1.0)), # Régularisation RECOMMANDÉE
])

pipe_poly.fit(X_train, y_train)
print('RMSE test :', np.sqrt(mean_squared_error(y_test,
pipe_poly.predict(X_test))))

# Nombre de features générées
n_feat_poly = pipe_poly.named_steps['poly'].n_output_features_
print(f'Features générées : {n_feat_poly}')
```

5.3 Le piège du degré trop élevé — overfitting polynomial

Augmenter le degré augmente la flexibilité du modèle. À un certain point, le modèle passe exactement par tous les points d'entraînement (interpolation de Runge) mais oscille violemment entre eux — il généralise très mal.

Degré	Comportement	RMSE train	RMSE test	Diagnostic
1 (linéaire)	Droite — trop simple	Élevé	Élevé	Underfitting (biais fort)
3–5	Courbe souple — bien adapté	Moyen	Faible	Bon compromis
10+	Courbe très oscillante	Très faible	Très élevé	Overfitting (variance forte)
n (nb obs.)	Passe par tous les points	≈ 0	Catastrophique	Overfitting maximal

La règle : toujours associer PolynomialFeatures à une régularisation

Avec un degré élevé, le nombre de features explose. Sans régularisation, les coefficients deviennent instables et énormes. Ridge ou Lasso est indispensable.

Résumé des bonnes pratiques pour la régression polynomiale

Commencer toujours par `degree=2` et valider par cross-validation avant d'augmenter.

Associer systématiquement à un Ridge ou Lasso : `PolynomialFeatures` → `StandardScaler` → Ridge.

Ne pas oublier `include_bias=False` si le Pipeline contient déjà un intercept (`LinearRegression`, `Ridge`).

Attention à l'explosion du nombre de features : $p \text{ features} \times \text{degré } d \rightarrow C(p+d, d) \text{ features}$. Avec $p=10$ et $d=3$: 286 features !

Valider le degré optimal par cross-validation, pas en observant le RMSE train seul.



Synthèse

Formules, tableau de décision et checklist

Tableau de décision : quel modèle utiliser ?

Situation	Modèle recommandé	Pourquoi
y continu, relation linéaire, $p \ll n$	OLS / Ridge / Lasso	Interprétable, rapide, bien maîtrisé
y continu, relation non-linéaire modérée	Polynomiale + Ridge	Flexibilité contrôlée sans changer d'algorithme
y continu, relation complexe	Arbre de régression	Capture interactions et non-linéarités automatiquement
y binaire (0/1), frontière linéaire	Régression logistique	Probabilités calibrées, interprétable, rapide
y binaire, frontière non-linéaire	Arbre + régression logistique + features poly	À comparer par CV
Dataset déséquilibré (< 5% positifs)	Logistique + F1 / AUC-PR	Accuracy trompeuse, utiliser F1 ou AUC Précision-Rappel

Formules clés du Cours 3

Formule	Expression	À retenir
Sigmoïde	$\sigma(z) = 1 / (1 + e^{(-z)})$	Maps $\mathbb{R} \rightarrow (0,1)$ · $\sigma(0) = 0.5$
Log-odds	$\log(p/(1-p)) = \beta_0 + \sum \beta_j x_j$	Interprétation des coefs : $\exp(\beta_j) = \text{odds ratio}$
Log Loss	$-(1/n) \cdot \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$	Fonction coût pour la classification binaire
Précision	$TP / (TP + FP)$	Parmi les prédits positifs, vrais positifs
Rappel	$TP / (TP + FN)$	Parmi les vrais positifs, détectés
F1-Score	$2 \cdot P \cdot R / (P + R)$	Compromis précision/rappel
AUC	$P(\text{score}^+ > \text{score}^-)$	0.5 = aléatoire · 1.0 = parfait
Poly degré d	$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d$	Toujours associer à Ridge ou Lasso